

Traffic Rule Violation Detection for Two-Wheelers Using a Lightweight Multi-Stage Vision Pipeline

Vishal Reddy K
Computer Vision Course Project

Abstract—This paper presents an offline computer vision system for detecting motorcycle traffic violations in a single RGB street image. The target problem is to identify overloaded motorcycles, riders without helmets, and the associated license plate for each violating vehicle. The proposed solution uses a lightweight multi-stage pipeline: a YOLO-based detector for motorcycles, persons, helmets, and plates; a geometric associator that links riders and plates to each motorcycle; and an EasyOCR-based license plate reader with custom preprocessing and post-processing. The final deployment stays within the 250 MB model-size limit and produces the required JSON output format for downstream evaluation.

Index Terms—traffic violation detection, motorcycle safety, YOLO, license plate OCR, object association, computer vision

I. INTRODUCTION

Traffic enforcement in dense urban scenes requires more than object detection. A useful system must determine whether a motorcycle is carrying too many riders, whether the riders are wearing helmets, and which registration plate belongs to the violating vehicle. This task is difficult because riders are frequently occluded, helmets are small objects, and license plates are often blurry or partially visible.

The implemented pipeline is designed for a constrained setting: a single input image, offline inference, and a strict model budget. The system must return only violating motorcycles and report the number of riders, the number of helmet violations, and the recognized plate text when available.

II. PROBLEM STATEMENT

Given one RGB street image, detect every motorcycle that violates traffic rules and output the result in the required structured format. A motorcycle is considered violating if either of the following conditions holds:

- the motorcycle carries more than two riders, or
- at least one associated rider is not wearing a helmet.

For each violating motorcycle, the system should also infer the license plate text when a plate can be localized and read. The report must remain valid even in crowded scenes with occlusion, clutter, and small objects.

III. PROPOSED METHODOLOGY

A. Overall Pipeline

The implementation follows a multi-stage design rather than a single monolithic model. The main detector predicts motorcycles, persons, helmets, and license plates. A separate base YOLO model provides robust motorcycle and person

detections, while the custom combined detector contributes helmet and plate predictions. The two detection streams are merged before association.

B. Detection and Class Mapping

The project uses a fixed class mapping across training, inference, and auditing:

Class ID	Semantic Class
0	motorcycle
1	person
2	helmet
3	license_plate

This consistency is critical because the association and violation logic assumes the same labels throughout the pipeline.

C. Motorcycle-First Association

The main challenge is not raw detection, but assigning detected persons, helmets, and plates to the correct motorcycle. Pure IoU is unreliable for riders because a seated passenger may extend beyond the bike box. The associator therefore uses three geometric rules in priority order:

- 1) **Containment:** a person is assigned to a motorcycle if a large fraction of the person box overlaps the motorcycle box.
- 2) **Proximity:** if containment fails, the bottom-center of the person box is compared with the motorcycle location to capture riders whose boxes protrude above the bike.
- 3) **IoU fallback:** a relaxed IoU threshold is used as a final safety net for partially visible motorcycles.

Helmet association is handled separately by comparing each helmet with the upper portion of each person box. This reflects the expected spatial relationship between a helmet and the rider's head. Plates are associated using an expanded motorcycle box to allow for plates that sit slightly below the bike body.

D. Violation Decision Logic

For each motorcycle group, the system computes the number of riders and helmet count. A motorcycle is flagged as violating if

$$\text{violation} = (\text{num_riders} > 2) \vee (\text{helmet_violations} > 0). \quad (1)$$

Only violating motorcycles are passed to OCR. This keeps the final output focused on the required cases and avoids unnecessary plate reading for compliant vehicles.

TABLE I
MODEL AUDIT AND DEPLOYMENT SUMMARY

Component	Size (MB)
combined_detector.pt	51.2
yolo11s.pt	19.3
yolo11l.pt	51.4
yolov8s.pt	22.6
Total	144.5

E. License Plate OCR

License plate recognition uses EasyOCR with a plate-specific preprocessing pipeline. The crop is resized when needed, converted to grayscale, enhanced with CLAHE, and sharpened using an adaptive blur-based filter. The OCR output is then normalized by removing noise characters, correcting common confusions such as O/0, I/1, S/5, and validating the result with a relaxed Indian plate pattern.

If the strict pattern check fails, the cleaned text is still retained instead of being discarded. This is useful for damaged or partially visible plates.

IV. IMPLEMENTATION DETAILS

The codebase is intentionally modular. The public entry point is `TrafficViolationDetector` in `solution.py`. This class loads the main YOLO detector, the optional base model for motorcycles, and the OCR reader once during initialization. The `predict()` method is stateless and can be called repeatedly on different images.

The actual pipeline is implemented in three layers. `pipeline/detector.py` converts YOLO outputs into typed detections. `pipeline/associator.py` groups detections by motorcycle using geometry-only rules. `pipeline/ocr.py` reads license plates only for bikes that are already classified as violating.

A. Training and Deployment

The project also includes training utilities for dataset preparation and model export. The combined dataset merges motorcycle, person, helmet, and plate annotations into a single class space. The deployment audit script checks class labels, model sizes, and CPU inference speed before submission.

B. Why the Code Structure Matters

This structure keeps the pipeline testable. The association logic can be unit tested without weights, the OCR layer can be exercised independently, and the final API remains simple enough for automated grading. The code therefore separates detection, grouping, and OCR rather than mixing them into one large inference loop.

V. EXPERIMENTAL RESULTS

The project was audited using the provided model directory. The key deployment results are summarized in Table I.

The class mapping audit confirmed that all four required labels are correctly aligned with the model: motorcycle, person,

TABLE II
EXAMPLE VIOLATION OUTPUT

Field	Example value
num_riders	3
helmet_violations	2
license_plate	MH12AB1234



Fig. 1. Best violation example: three riders on one motorcycle, all localized by the detector.

helmet, and license plate. On CPU, the end-to-end detector averaged 533 ms per image with a p95 of 558 ms per image. This is suitable for single-image inference, but not real-time video.

Representative output from the required JSON interface is summarized in Table II.

This example shows an overloaded motorcycle with two unhelmeted riders and a readable plate string.

VI. VISUAL EXAMPLES

The report includes the strongest visual results from the workspace. Figure 1 shows a clear triple-riding violation with the rider boxes, while Figure 2 shows a successful license plate read with the detected plate region.

VII. CODE EXCERPTS

The following abridged excerpts show the main control flow of the implementation.

A. Public Inference API

Listing 1. Main prediction flow in `solution.py`

```
def predict(self, image_path: str) -> dict[str, Any]:
    image = self._load_image(image_path)
    if image is None:
        return {"violations": []}

    detections = self._detector.detect(image)
    detections.persons.clear()
    detections.helmets = [
        h for h in detections.helmets if h.conf >=
        0.40
```



Fig. 2. Best OCR example: the system localizes the plate and returns the registration number.

```

    ]
    detections.plates = [
        p for p in detections.plates if p.conf >=
            0.40
    ]

    rider_groups = associate(detections)
    violations = []
    for group in rider_groups:
        if not group.is_violation:
            continue
        plate_text = ""
        if group.best_plate is not None:
            plate_text = self._ocr.read(image,
                group.best_plate.bbox)
        violations.append(
            {
                "num_riders": group.num_riders,
                "helmet_violations":
                    group.helmet_violations,
                "license_plate": plate_text,
            }
        )
    return {"violations": violations}

```

B. Motorcycle Association

Listing 2. Geometry-first rider association in associator.py

```

def associate_persons_to_moto(
    moto: Detection,
    persons: list[Detection],
) -> list[Detection]:

```

```

associated = []
for person in persons:
    ratio = person.overlap_ratio(moto)
    if ratio >= PERSON_IN_MOTO_RATIO:
        associated.append(person)
        continue

    prox_thresh = moto.height *
        PERSON_PROX_FACTOR
    dx = abs(person.bottom_cx - moto.cx)
    dy = abs(person.bottom_cy - moto.cy)
    if dx < moto.width * 0.6 and dy <
        prox_thresh:
        associated.append(person)
        continue

    if person.iou(moto) >
        PERSON_MOTO_IU_THRESHOLD:
        associated.append(person)

return associated

def associate(detections: FrameDetections) ->
    list[RiderGroup]:
    claimed_persons: set[int] = set()
    groups: list[RiderGroup] = []
    for moto in detections.motorcycles:
        candidates = [
            p for p in detections.persons if
                id(p) not in claimed_persons
        ]
        persons = associate_persons_to_moto(moto,
            candidates)
        for p in persons:
            claimed_persons.add(id(p))
        person_helmet_map =
            associate_helmets_to_persons(
                persons, detections.helmets
            )
        all_helmets = []
        for lst in person_helmet_map.values():
            all_helmets.extend(lst)
        plates = associate_plates_to_moto(moto,
            detections.plates)
        groups.append(
            RiderGroup(
                motorcycle=moto,
                persons=persons,
                helmets=all_helmets,
                plates=plates,
            )
        )
    return groups

```

C. OCR and Post-Processing

Listing 3. Plate OCR preprocessing and cleanup in ocr.py

```

def read(
    self,
    image: np.ndarray,
    plate_box: tuple[float, float, float, float] |
        None = None,
) -> str:
    alnum = "ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789"
    crop = self._crop(image, plate_box)
    if crop is None or crop.size == 0:
        return ""

    processed = _preprocess_plate_crop(crop)
    raw_results = self._reader.readtext(
        processed,
        detail=0,
        allowlist=alnum,
        paragraph=False,
        width_ths=0.7,
        mag_ratio=1.5,
    )

    if not raw_results:
        raw_results = self._reader.readtext(
            crop,

```

```

        detail=0,
        allowlist=alnum,
        paragraph=False,
    )

    if not raw_results:
        return ""

    combined = "".join(raw_results).upper()
    cleaned = _clean_plate(combined)
    fixed = _apply_positional_fix(cleaned)
    if _PLATE_PATTERN.match(fixed):
        return fixed
    return cleaned if cleaned else ""

```

VIII. DISCUSSION

The code excerpts show that the pipeline is not a generic object detector. It is a structured decision system that first localizes motorcycles, then groups riders and helmets, and finally performs OCR only on violating groups. This staged design is what makes the solution reliable enough for the constrained project setting.

IX. LIMITATIONS

The system is effective for the intended assignment, but several limitations remain.

- **CPU latency:** inference is slower on CPU than on GPU, and the current settings are tuned for reliability rather than maximum speed.
- **Occlusion sensitivity:** severe overlap between riders, motorcycles, and surrounding traffic can still cause missed associations.
- **Helmet confusion:** helmets may be confused with caps, turbans, or other small headwear-like shapes in difficult scenes.
- **OCR fragility:** plate reading remains sensitive to blur, motion, extreme perspective, and strong compression artifacts.
- **Dependency on detection quality:** if the upstream detector misses a person or helmet, the downstream violation logic cannot recover that error.

X. CONCLUSION

The proposed system provides a compact and practical solution for motorcycle traffic violation detection. Its main strengths are the multi-stage design, the geometric association strategy, and the OCR cleanup pipeline, all of which are tailored to the specific problem constraints. The audited deployment remains within the model-size budget and produces the expected structured output. Future work should focus on stronger small-object detection, better OCR robustness under blur, and faster CPU inference.

REFERENCES

- [1] Ultralytics, “YOLO Documentation,” <https://docs.ultralytics.com/>.
- [2] JaidedAI, “EasyOCR,” <https://github.com/JaidedAI/EasyOCR>.
- [3] OpenCV, “OpenCV Library,” <https://opencv.org/>.